# PL/SQL Exercises - Solutions

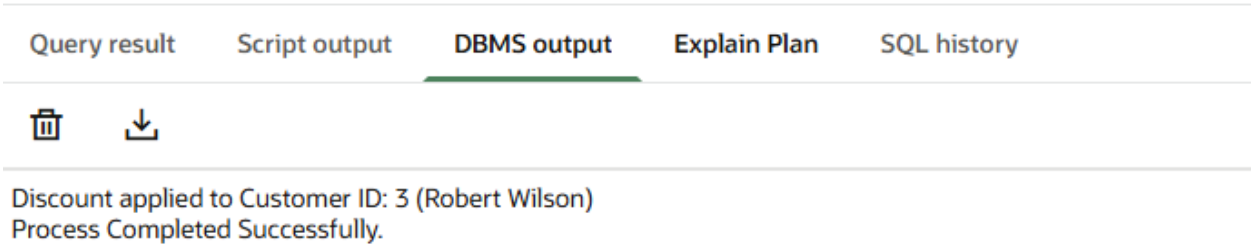
## Exercise 1: Control Structures

### Scenario 1: Interest Rate Discount for Senior Customers

Apply a 1% discount to loan interest rates for customers above 60 years old.

```
BEGIN

  -- Loop through all customers who have loans
  FOR cust_rec IN (
    SELECT c.CustomerID,
           c.Name,
           c.DOB,
           l.LoanID,
           l.InterestRate
    FROM Customers c
    JOIN Loans l
      ON c.CustomerID = l.CustomerID
  )
  LOOP

    -- Check if customer's age is greater than 60 years
    IF FLOOR(MONTHS_BETWEEN(SYSDATE, cust_rec.DOB) / 12) > 60 THEN

      -- Apply a 1% discount on the current interest rate
      UPDATE Loans
      SET InterestRate = InterestRate - (InterestRate * 0.01)
      WHERE LoanID = cust_rec.LoanID;

      -- Displays message showing which customer received the discount
      DBMS_OUTPUT.PUT_LINE(
        'Discount applied to Customer ID: '
        || cust_rec.CustomerID
        || ' (' || cust_rec.Name || ')'
      );

    END IF;

  END LOOP;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Process Completed Successfully.');

END;
/
```

**OUTPUT:**



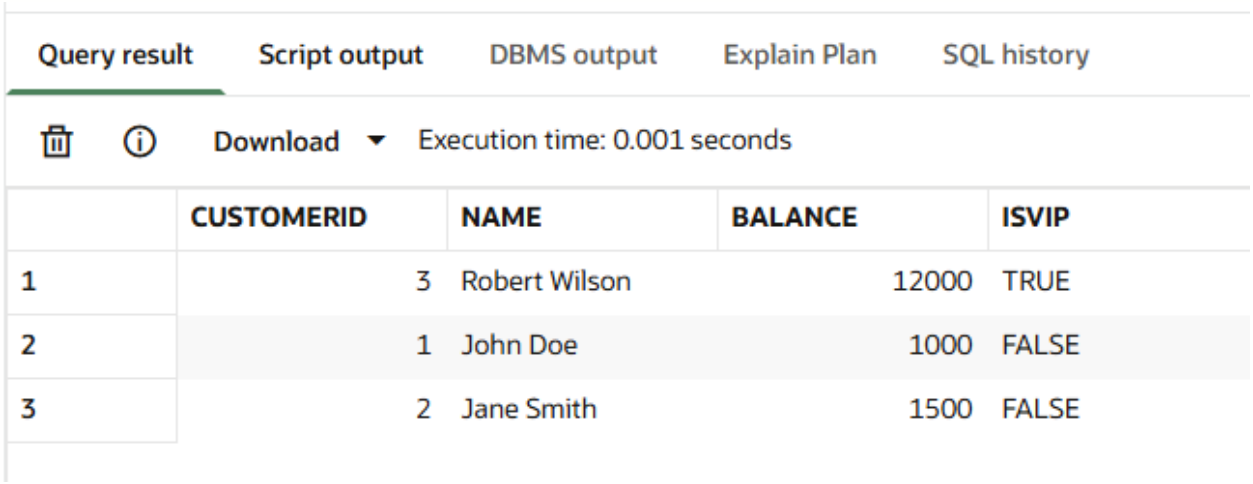

## Scenario 2: VIP Status Based on Balance

Set IsVIP flag to TRUE for customers with balance over $10,000.

```
-- Add a new column to store VIP status (Run only once)
ALTER TABLE Customers
ADD IsVIP VARCHAR2(5) DEFAULT 'FALSE';

BEGIN

  -- Loop through all customers
  FOR cust_rec IN (
    SELECT CustomerID,
           Name,
           Balance
    FROM Customers
  )
  LOOP

    -- Check if customer's balance is greater than 10000
    IF cust_rec.Balance > 10000 THEN

      -- Update VIP status to TRUE
      UPDATE Customers
      SET IsVIP = 'TRUE'
      WHERE CustomerID = cust_rec.CustomerID;

      -- Display customer who became VIP
      DBMS_OUTPUT.PUT_LINE(
        'VIP Status Granted to Customer ID: '
        || cust_rec.CustomerID
        || ' (' || cust_rec.Name || ')'
      );

    END IF;

  END LOOP;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('VIP Status Update Completed Successfully.');
```

```
END;
/

-- Display updated customer details
SELECT CustomerID,
       Name,
       Balance,
       IsVIP
FROM Customers;
```

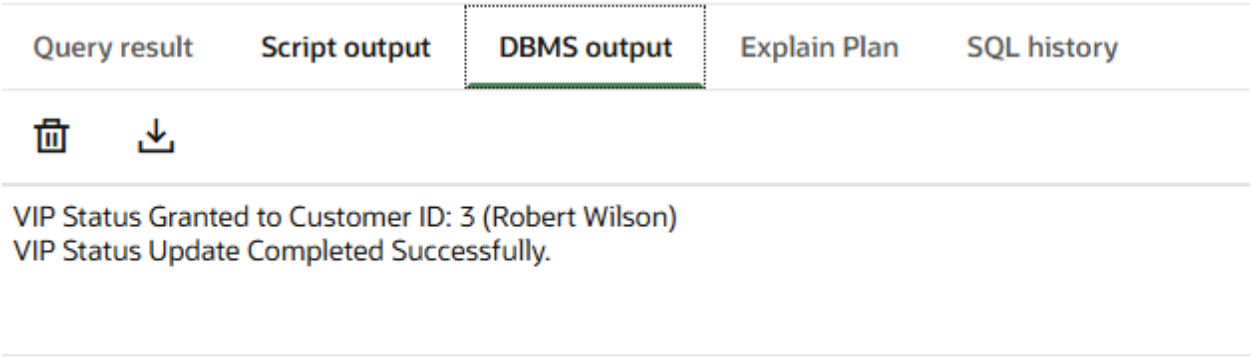
**OUTPUT**:



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

| | CUSTOMERID | NAME | BALANCE | ISVIP |
|---|---|---|---|---|
| 1 | 3 | Robert Wilson | 12000 | TRUE |
| 2 | 1 | John Doe | 1000 | FALSE |
| 3 | 2 | Jane Smith | 1500 | FALSE |



Query result | Script output | DBMS output | Explain Plan | SQL history

```
VIP Status Granted to Customer ID: 3 (Robert Wilson)
VIP Status Update Completed Successfully.
```

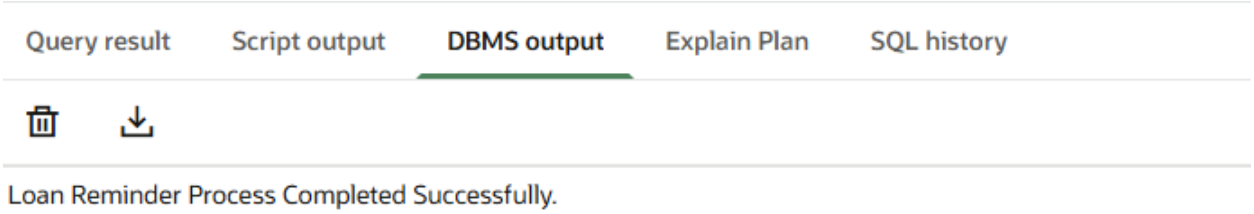
## Scenario 3: Loan Due Reminders

Print a reminder for each loan due within the next 30 days.

```
DECLARE
  v_message VARCHAR2(200);

BEGIN

  -- Loop through all loans due within the next 30 days
  FOR loan_rec IN (
```

```
    SELECT l.LoanID,
           l.CustomerID,
           l.EndDate,
           c.Name
    FROM Loans l
    JOIN Customers c
      ON l.CustomerID = c.CustomerID
    WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
  )
  LOOP

    -- Create and display the reminder message
    v_message := 'Reminder: Loan ID ' || loan_rec.LoanID ||
                 ' for customer ' || loan_rec.Name ||
                 ' is due on ' || TO_CHAR(loan_rec.EndDate, 'DD-MON-YYYY');

    DBMS_OUTPUT.PUT_LINE(v_message);

  END LOOP;

  DBMS_OUTPUT.PUT_LINE('Loan Reminder Process Completed Successfully.');

END;
/
```

**OUTPUT:**




# Exercise 2: Error Handling

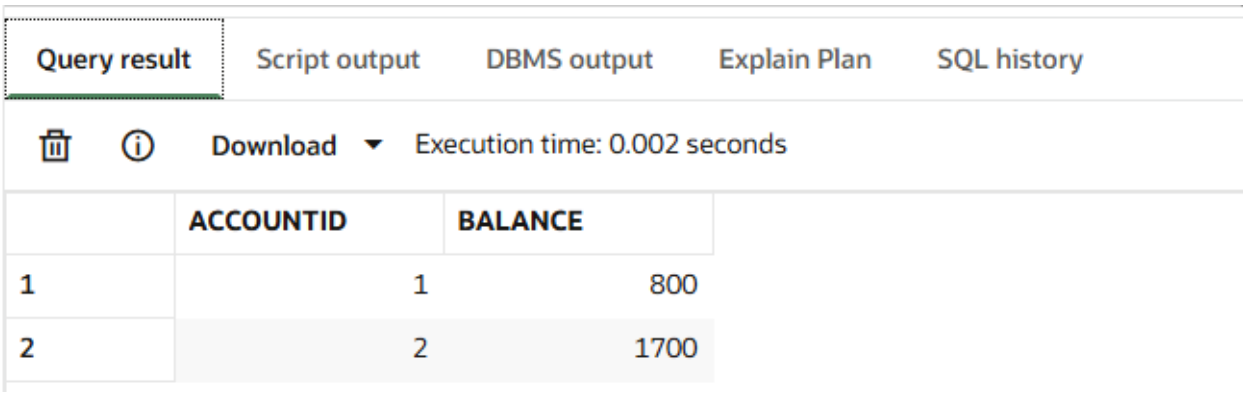
## Scenario 1: SafeTransferFunds

Transfer funds between two accounts and roll back safely on error.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (
  p_from_account IN NUMBER,
  p_to_account   IN NUMBER,
  p_amount       IN NUMBER
) AS

  v_from_balance NUMBER;

BEGIN

  -- Get the balance of the source account
```

```
  SELECT Balance
  INTO v_from_balance
  FROM Accounts
  WHERE AccountID = p_from_account
  FOR UPDATE;

  -- Check if sufficient balance is available
  IF v_from_balance < p_amount THEN
    RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in source account');
  END IF;

  -- Deduct amount from source account
  UPDATE Accounts
  SET Balance = Balance - p_amount
  WHERE AccountID = p_from_account;

  -- Add amount to destination account
  UPDATE Accounts
  SET Balance = Balance + p_amount
  WHERE AccountID = p_to_account;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Funds Transferred Successfully.');

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error during fund transfer: ' || SQLERRM);

END;
/

BEGIN
  -- Call the procedure to transfer 200 from Account 1 to Account 2
  SafeTransferFunds(1, 2, 200);
END;
/

-- Display the updated balances of all accounts
SELECT AccountID,
       Balance
FROM Accounts;
```

**OUTPUT**:




| | ACCOUNTID | BALANCE |
|---|---|---|
| 1 | 1 | 800 |
| 2 | 2 | 1700 |

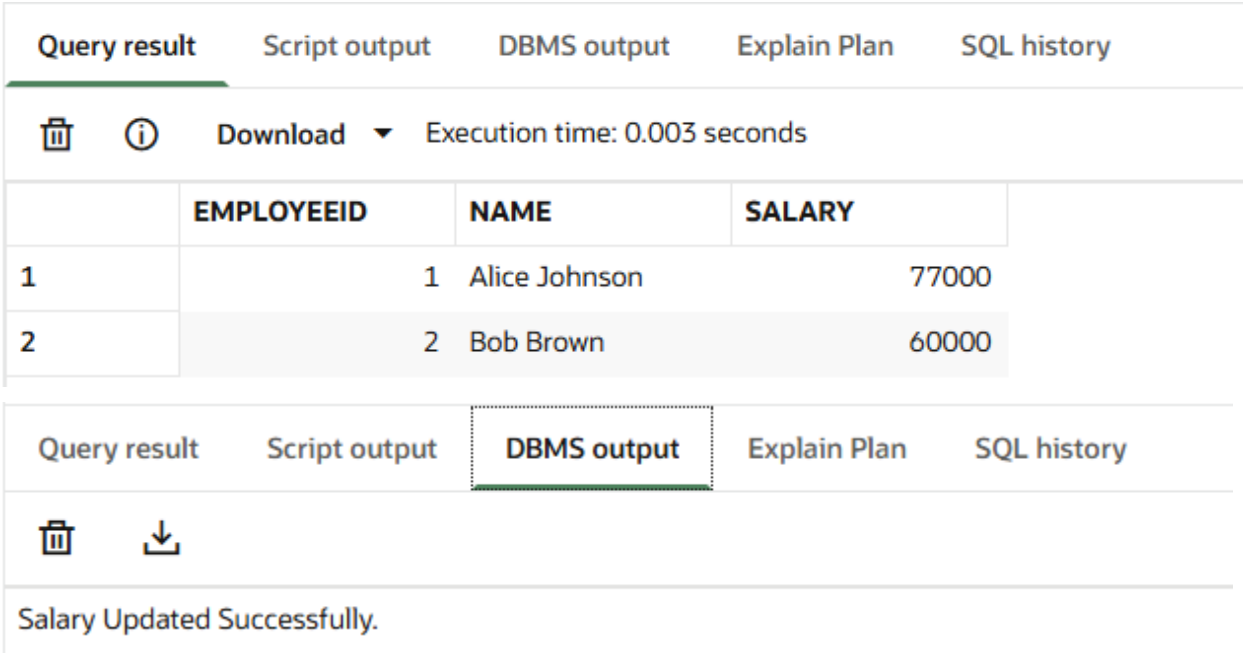
## Scenario 2: UpdateSalary

Increase salary by a given percentage, handling a missing employee ID.

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
  p_employee_id IN NUMBER,
  p_percentage  IN NUMBER
) AS

  v_count NUMBER;

BEGIN

  -- Check whether the employee exists
  SELECT COUNT(*)
  INTO v_count
  FROM Employees
  WHERE EmployeeID = p_employee_id;

  IF v_count = 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Employee ID does not exist');
  END IF;

  -- Increase the employee's salary
  UPDATE Employees
  SET Salary = Salary + (Salary * p_percentage / 100)
  WHERE EmployeeID = p_employee_id;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully.');

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error updating salary: ' || SQLERRM);

END;
```

```
/

BEGIN
  -- Call the procedure to increase Employee 1's salary by 10%
  UpdateSalary(1, 10);
END;
/

-- Display the updated employee details
SELECT EmployeeID,
       Name,
       Salary
FROM Employees;
```

**OUTPUT**:



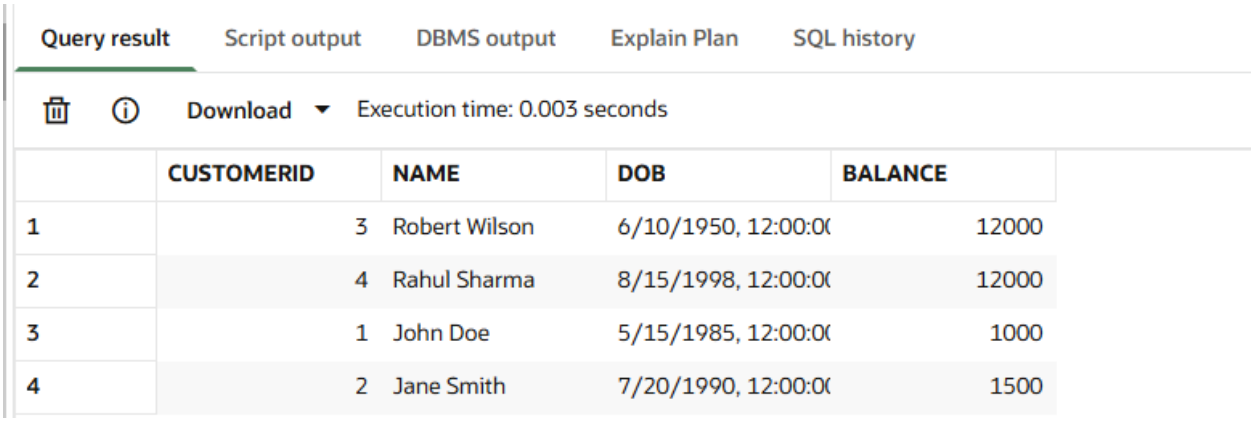

## Scenario 3: AddNewCustomer

Insert a new customer and prevent duplicate customer IDs.

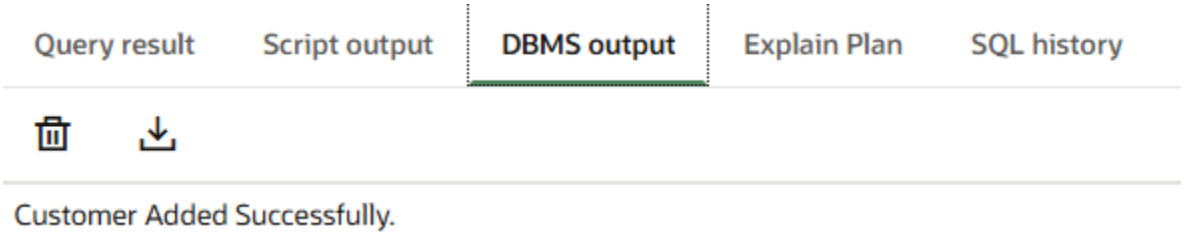
```
CREATE OR REPLACE PROCEDURE AddNewCustomer (
  p_customer_id IN NUMBER,
  p_name        IN VARCHAR2,
  p_dob         IN DATE,
  p_balance     IN NUMBER
) AS

BEGIN

  -- Insert a new customer into the Customers table
  INSERT INTO Customers (
    CustomerID,
    Name,
```

```
    DOB,
    Balance,
    LastModified
  )
  VALUES (
    p_customer_id,
    p_name,
    p_dob,
    p_balance,
    SYSDATE
  );

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Customer Added Successfully.');

EXCEPTION

  -- Handle duplicate customer ID
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Error: Customer with this ID already exists');

  -- Handle other errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error adding customer: ' || SQLERRM);

END;
/

BEGIN
  -- Call the procedure to add a new customer
  AddNewCustomer(
    4,
    'Rahul Sharma',
    TO_DATE('1998-08-15','YYYY-MM-DD'),
    12000
  );
END;
/

-- Display the updated customer details
SELECT CustomerID,
       Name,
       DOB,
       Balance
FROM Customers;
```

**OUTPUT:**




| | CUSTOMERID | NAME | DOB | BALANCE |
|---|---|---|---|---|
| 1 | 3 | Robert Wilson | 6/10/1950, 12:00:0( | 12000 |
| 2 | 4 | Rahul Sharma | 8/15/1998, 12:00:0( | 12000 |
| 3 | 1 | John Doe | 5/15/1985, 12:00:0( | 1000 |
| 4 | 2 | Jane Smith | 7/20/1990, 12:00:0( | 1500 |




# Exercise 3: Stored Procedures

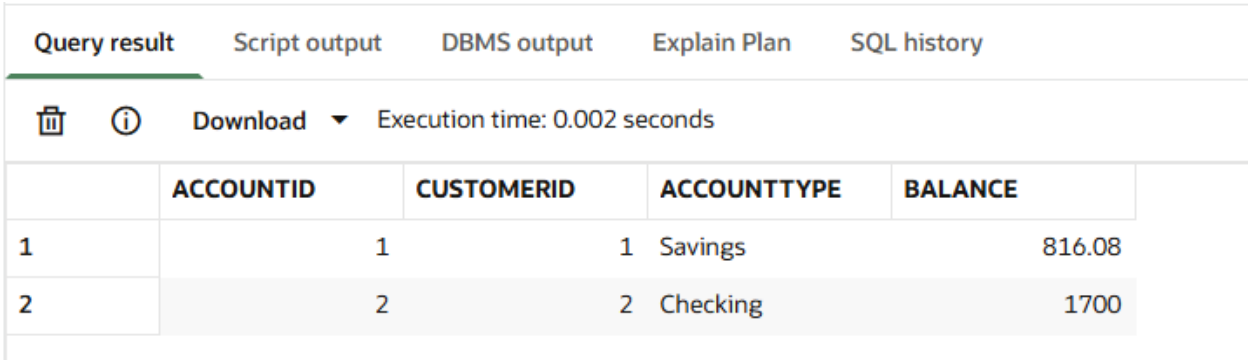
## Scenario 1: ProcessMonthlyInterest

Apply a 1% interest rate to all savings account balances.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS

BEGIN

  -- Apply 1% monthly interest to all savings accounts
  UPDATE Accounts
  SET Balance = Balance + (Balance * 0.01),
      LastModified = SYSDATE
  WHERE AccountType = 'Savings';

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Monthly Interest Processed Successfully.');

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error processing monthly interest: ' || SQLERRM);

END;
/
```

```
BEGIN
  -- Call the procedure to process monthly interest
  ProcessMonthlyInterest;
END;
/

-- Display the updated account details
SELECT AccountID,
       CustomerID,
       AccountType,
       Balance
FROM Accounts;
```

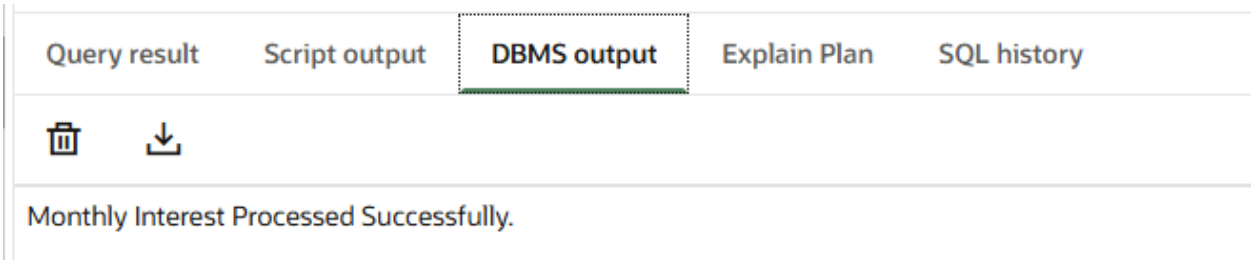
**OUTPUT**:



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.002 seconds

| | ACCOUNTID | CUSTOMERID | ACCOUNTTYPE | BALANCE |
|---|---|---|---|---|
| 1 | 1 | 1 | Savings | 816.08 |
| 2 | 2 | 2 | Checking | 1700 |



Query result | Script output | DBMS output | Explain Plan | SQL history

Monthly Interest Processed Successfully.

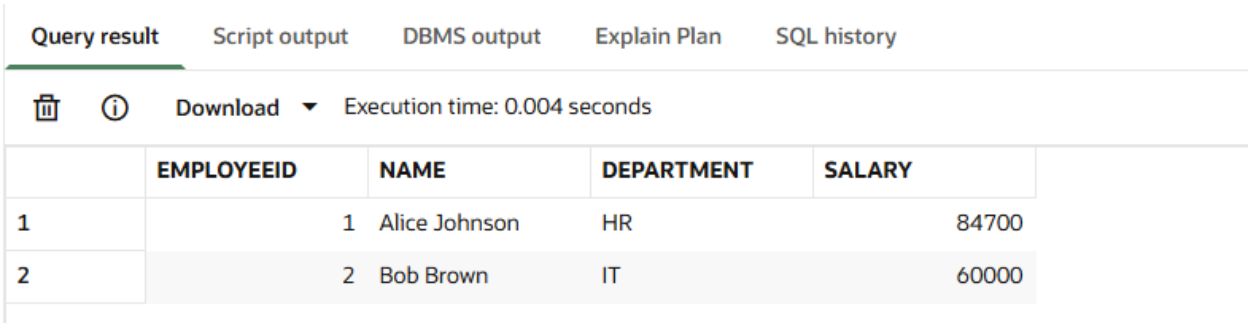
## Scenario 2: UpdateEmployeeBonus

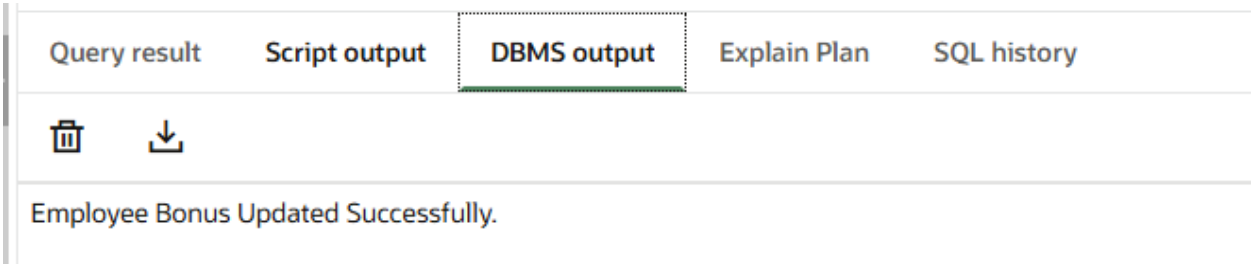
Add a bonus percentage to salaries of employees in a given department.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
  p_department IN VARCHAR2,
  p_bonus_pct  IN NUMBER
) AS

BEGIN

  -- Update the salary of employees in the given department
  UPDATE Employees
  SET Salary = Salary + (Salary * p_bonus_pct / 100)
  WHERE Department = p_department;

  -- Save all changes
```

```
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Employee Bonus Updated Successfully.');

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error updating employee bonus: ' || SQLERRM);

END;
/

BEGIN
  -- Call the procedure to add a 10% bonus for the HR department
  UpdateEmployeeBonus('HR', 10);
END;
/

-- Display the updated employee details
SELECT EmployeeID,
       Name,
       Department,
       Salary
FROM Employees;
```

**OUTPUT:**




## Scenario 3: TransferFunds

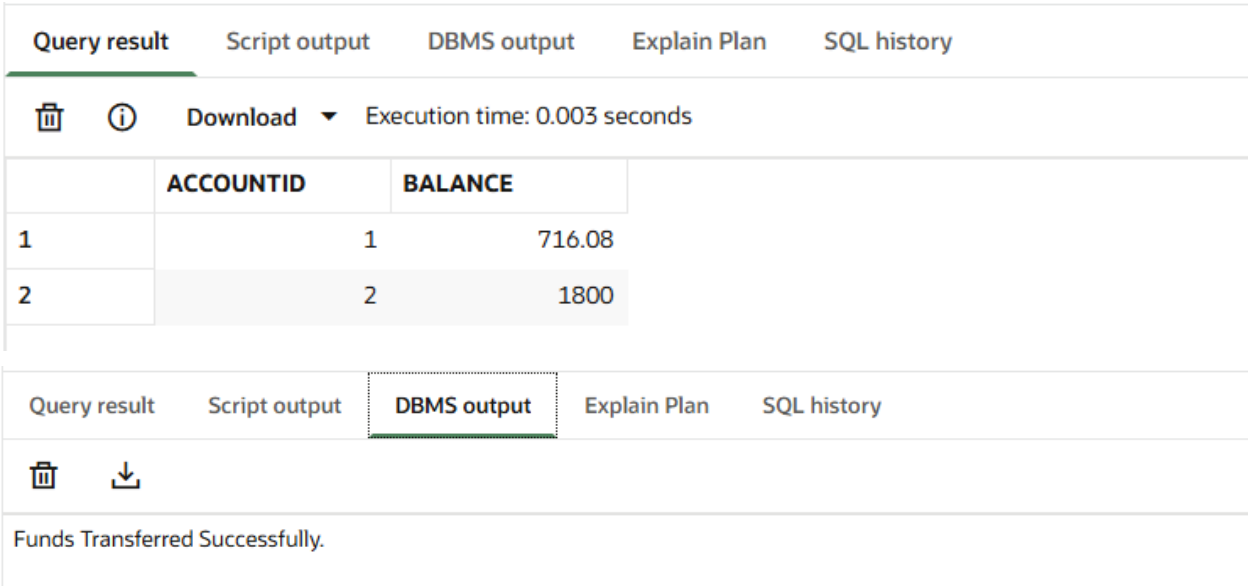
Transfer an amount between accounts after checking sufficient balance.

```
CREATE OR REPLACE PROCEDURE TransferFunds (
```

```
  p_from_account IN NUMBER,
  p_to_account   IN NUMBER,
  p_amount       IN NUMBER
) AS

  v_balance NUMBER;

BEGIN

  -- Get the balance of the source account
  SELECT Balance
  INTO v_balance
  FROM Accounts
  WHERE AccountID = p_from_account;

  -- Check if sufficient balance is available
  IF v_balance >= p_amount THEN

    -- Deduct amount from the source account
    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    -- Add amount to the destination account
    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    -- Save all changes
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds Transferred Successfully.');

  ELSE

    DBMS_OUTPUT.PUT_LINE('Transfer Failed: Insufficient Balance.');

  END IF;

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error during transfer: ' || SQLERRM);

END;
/

BEGIN
  -- Call the procedure to transfer 100 from Account 1 to Account 2
  TransferFunds(1, 2, 100);
END;
/
```

```
-- Display the updated balances of all accounts
SELECT AccountID,
       Balance
FROM Accounts;
```

**OUTPUT:**




# Exercise 4: Functions

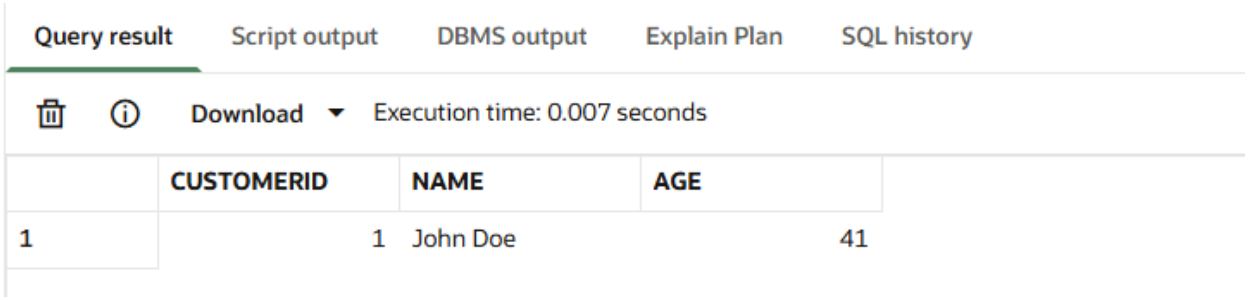
## Scenario 1: CalculateAge

Return a customer's age in years from their date of birth.

```
CREATE OR REPLACE FUNCTION CalculateAge (
  p_dob IN DATE
) RETURN NUMBER AS

  v_age NUMBER;

BEGIN

  -- Calculate the age in years
  v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);

  -- Return the calculated age
  RETURN v_age;

END;
/

-- Calculate the age of Customer ID 1
SELECT CustomerID,
       Name,
```

```
       CalculateAge(DOB) AS Age
FROM Customers
WHERE CustomerID = 1;
```

**OUTPUT:**



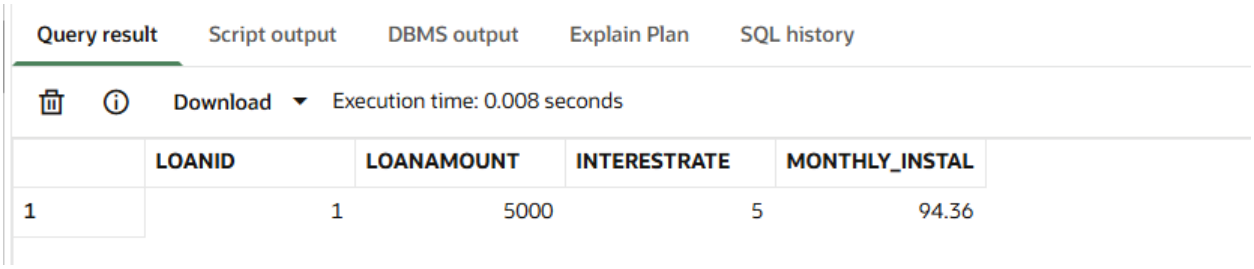

## Scenario 2: CalculateMonthlyInstallment

Return the monthly installment amount for a loan.

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (
  p_loan_amount   IN NUMBER,
  p_interest_rate IN NUMBER,
  p_years         IN NUMBER
) RETURN NUMBER AS

  v_monthly_rate NUMBER;
  v_months       NUMBER;
  v_emi          NUMBER;

BEGIN

  -- Calculate the monthly interest rate
  v_monthly_rate := (p_interest_rate / 100) / 12;

  -- Calculate the total number of monthly installments
  v_months := p_years * 12;

  -- Handle zero interest rate to avoid division by zero
  IF v_monthly_rate = 0 THEN
    v_emi := p_loan_amount / v_months;
  ELSE
    v_emi := (p_loan_amount * v_monthly_rate * POWER(1 + v_monthly_rate, v_months))
             / (POWER(1 + v_monthly_rate, v_months) - 1);
  END IF;

  -- Return the monthly installment amount
  RETURN ROUND(v_emi, 2);

END;
/

-- Calculate the monthly installment for Loan ID 1
SELECT LoanID,
```

```
       LoanAmount,
       InterestRate,
       CalculateMonthlyInstallment(LoanAmount, InterestRate, 5) AS Monthly_Installment
FROM Loans
WHERE LoanID = 1;
```

**OUTPUT:**




| | LOANID | LOANAMOUNT | INTERESTRATE | MONTHLY_INSTAL |
|---|---|---|---|---|
| 1 | 1 | 5000 | 5 | 94.36 |

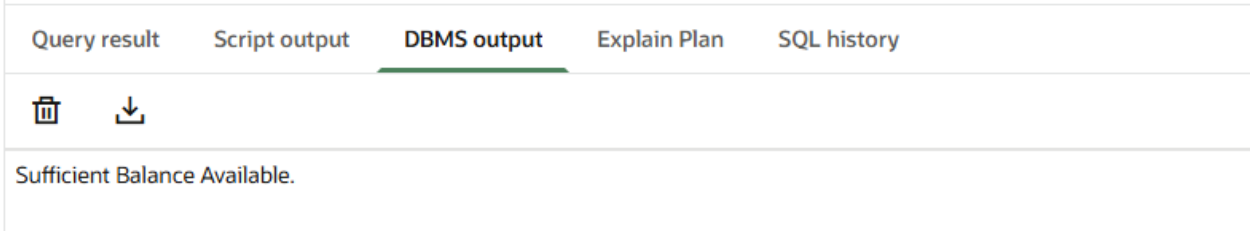
## Scenario 3: HasSufficientBalance

Return TRUE if the account balance is at least the specified amount.

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (
  p_account_id IN NUMBER,
  p_amount     IN NUMBER
) RETURN BOOLEAN AS

  v_balance NUMBER;

BEGIN

  -- Get the balance of the given account
  SELECT Balance
  INTO v_balance
  FROM Accounts
  WHERE AccountID = p_account_id;

  -- Check if the account has sufficient balance
  IF v_balance >= p_amount THEN
    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;

EXCEPTION

  -- Return FALSE if the account does not exist
  WHEN NO_DATA_FOUND THEN
    RETURN FALSE;

END;
/

DECLARE
```

```
  v_result BOOLEAN;

BEGIN

  -- Check whether Account 1 has at least 500 balance
  v_result := HasSufficientBalance(1, 500);

  IF v_result THEN
    DBMS_OUTPUT.PUT_LINE('Sufficient Balance Available.');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Insufficient Balance.');
  END IF;

END;
/
```

**OUTPUT:**




# Exercise 5: Triggers

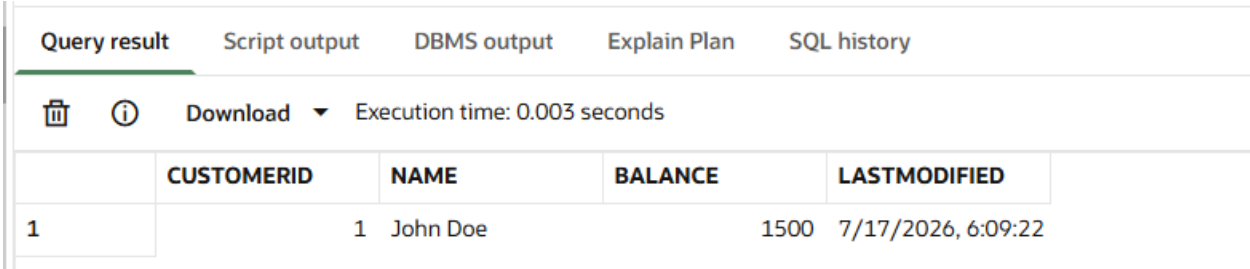
## Scenario 1: UpdateCustomerLastModified

Update LastModified whenever a customer record is updated.

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE ON Customers

FOR EACH ROW

BEGIN

  -- Automatically update LastModified date when customer details are updated
  :NEW.LastModified := SYSDATE;

END;
/

-- Update customer details to test the trigger
UPDATE Customers
SET Balance = Balance + 500
WHERE CustomerID = 1;

COMMIT;
```

```sql
-- Display updated customer details
SELECT CustomerID,
       Name,
       Balance,
       LastModified
FROM Customers
WHERE CustomerID = 1;
```

**OUTPUT:**




| | CUSTOMERID | NAME | BALANCE | LASTMODIFIED |
|---|---|---|---|---|
| 1 | 1 | John Doe | 1500 | 7/17/2026, 6:09:22 |

## Scenario 2: LogTransaction

Insert an audit log record whenever a transaction is added.

```sql
-- Create table to store the audit trail
CREATE TABLE AuditLog (
    LogID            NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    TransactionID    NUMBER,
    AccountID        NUMBER,
    Amount           NUMBER,
    TransactionType  VARCHAR2(10),
    LogDate          DATE
);

CREATE OR REPLACE TRIGGER LogTransaction

AFTER INSERT ON Transactions
FOR EACH ROW

BEGIN

  -- Insert transaction details into audit log
  INSERT INTO AuditLog (
    TransactionID,
    AccountID,
    Amount,
    TransactionType,
    LogDate
  )
  VALUES (
    :NEW.TransactionID,
    :NEW.AccountID,
    :NEW.Amount,
    :NEW.TransactionType,
    SYSDATE
```

```
  );

END;
/

-- Insert a new transaction to test the trigger
INSERT INTO Transactions (
    TransactionID,
    AccountID,
    TransactionDate,
    Amount,
    TransactionType
)
VALUES (
    3,
    1,
    SYSDATE,
    500,
    'Deposit'
);

COMMIT;

-- Display the audit log details
SELECT *
FROM AuditLog;
```

**OUTPUT:**

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.001 seconds

| | LOGID | TRANSACTIONID | ACCOUNTID | AMOUNT | TRANSACTIONTYF | LOGDATE |
|---|---|---|---|---|---|---|
| 1 | 1 | 3 | 1 | 500 | Deposit | 7/17/2026, 6:15:04 |

## Scenario 3: CheckTransactionRules

Ensure withdrawals do not exceed balance and deposits are positive.

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW

DECLARE

  v_balance NUMBER;

BEGIN

  -- Get current account balance
  SELECT Balance
  INTO v_balance
  FROM Accounts
```

```
  WHERE AccountID = :NEW.AccountID;

  -- Check withdrawal rule
  IF :NEW.TransactionType = 'Withdrawal'
     AND :NEW.Amount > v_balance THEN

    RAISE_APPLICATION_ERROR(
      -20003,
      'Withdrawal amount exceeds account balance'
    );

  END IF;

  -- Check deposit rule
  IF :NEW.TransactionType = 'Deposit'
     AND :NEW.Amount <= 0 THEN

    RAISE_APPLICATION_ERROR(
      -20004,
      'Deposit amount must be positive'
    );

  END IF;

END;
/
```

# Exercise 6: Cursors

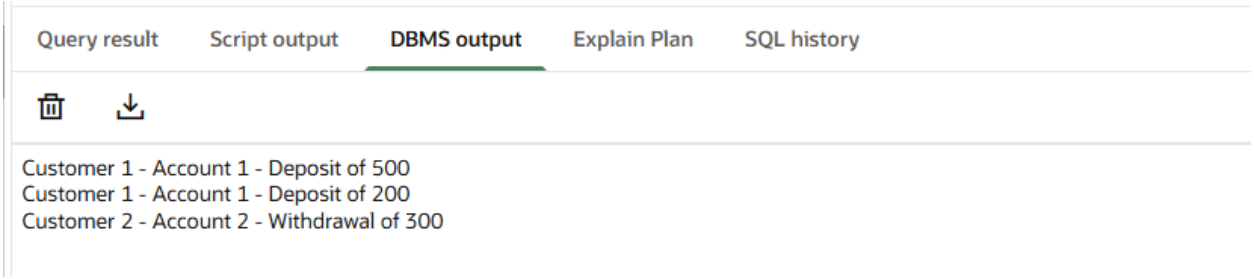
## Scenario 1: GenerateMonthlyStatements

Print a statement line for each transaction made in the current month.

```
DECLARE

  -- Cursor to fetch all transactions made in the current month
  CURSOR c_monthly_transactions IS
    SELECT t.TransactionID,
           t.AccountID,
           t.Amount,
           t.TransactionType,
           a.CustomerID
    FROM Transactions t
    JOIN Accounts a
    ON t.AccountID = a.AccountID
    WHERE EXTRACT(MONTH FROM t.TransactionDate) = EXTRACT(MONTH FROM SYSDATE)
    AND EXTRACT(YEAR FROM t.TransactionDate) = EXTRACT(YEAR FROM SYSDATE);

  v_statement VARCHAR2(300);

BEGIN

  -- Loop through each transaction
```

```
  FOR trans_rec IN c_monthly_transactions
  LOOP

    -- Generate customer transaction statement
    v_statement := 'Customer ' || trans_rec.CustomerID ||
                   ' - Account ' || trans_rec.AccountID ||
                   ' - ' || trans_rec.TransactionType ||
                   ' of ' || trans_rec.Amount;

    DBMS_OUTPUT.PUT_LINE(v_statement);

  END LOOP;

END;
/
```

**OUTPUT:**



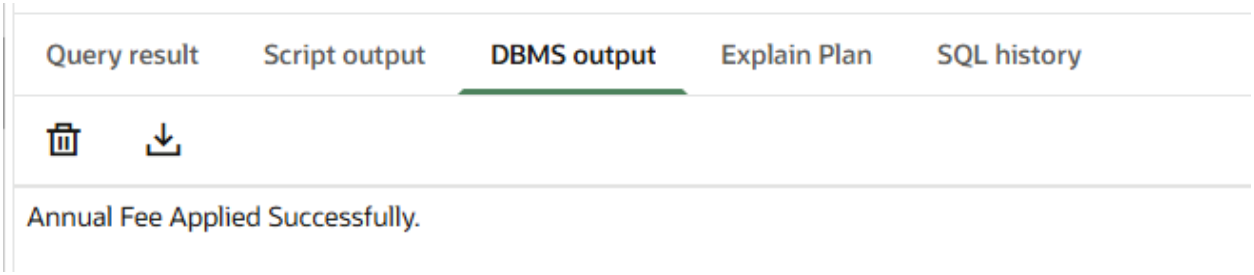

## Scenario 2: ApplyAnnualFee

Deduct a flat annual maintenance fee from every account.

```
DECLARE

  -- Cursor to fetch all account details
  CURSOR c_accounts IS
    SELECT AccountID,
           Balance
    FROM Accounts;

  v_fee NUMBER := 50;

BEGIN

  -- Loop through all accounts
  FOR acc_rec IN c_accounts
  LOOP

    -- Deduct annual maintenance fee from account balance
    UPDATE Accounts
    SET Balance = Balance - v_fee,
        LastModified = SYSDATE
    WHERE AccountID = acc_rec.AccountID;
```

```
  END LOOP;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Annual Fee Applied Successfully.');

END;
/

-- Display the updated account details
SELECT AccountID,
       Balance,
       LastModified
FROM Accounts;
```

**OUTPUT:**



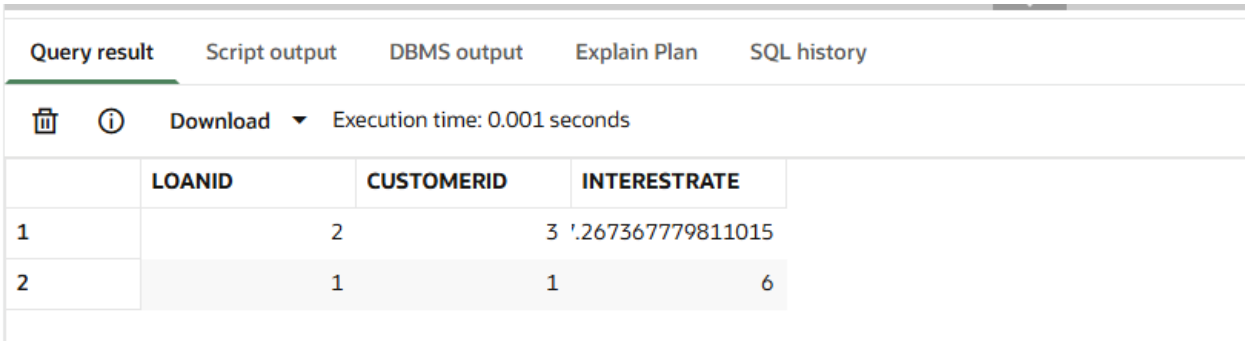

## Scenario 3: UpdateLoanInterestRates

Update interest rates for all loans based on a new policy.

```
DECLARE

  -- Cursor to fetch all loans
  CURSOR c_loans IS
    SELECT LoanID,
           InterestRate
    FROM Loans;

  v_new_rate NUMBER;

BEGIN

  -- Loop through all loans
  FOR loan_rec IN c_loans
  LOOP

    -- Apply new policy by adding 0.5% to current interest rate
    v_new_rate := loan_rec.InterestRate + 0.5;

    -- Update the loan interest rate
    UPDATE Loans
    SET InterestRate = v_new_rate
    WHERE LoanID = loan_rec.LoanID;
```

```
  END LOOP;

  -- Save all changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Loan Interest Rates Updated Successfully.');

END;
/

-- Display the updated loan interest rates
SELECT LoanID,
       CustomerID,
       InterestRate
FROM Loans;
```

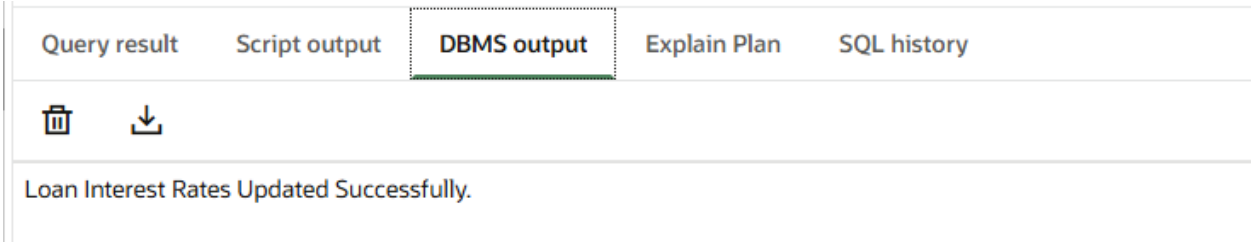
**OUTPUT:**



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

| | LOANID | CUSTOMERID | INTERESTRATE |
|---|---|---|---|
| 1 | 2 | 3 | '.267367779811015 |
| 2 | 1 | 1 | 6 |



Query result | Script output | DBMS output | Explain Plan | SQL history

Loan Interest Rates Updated Successfully.

# Exercise 7: Packages

## Scenario 1: CustomerManagement

Package with procedures and a function for customer records.

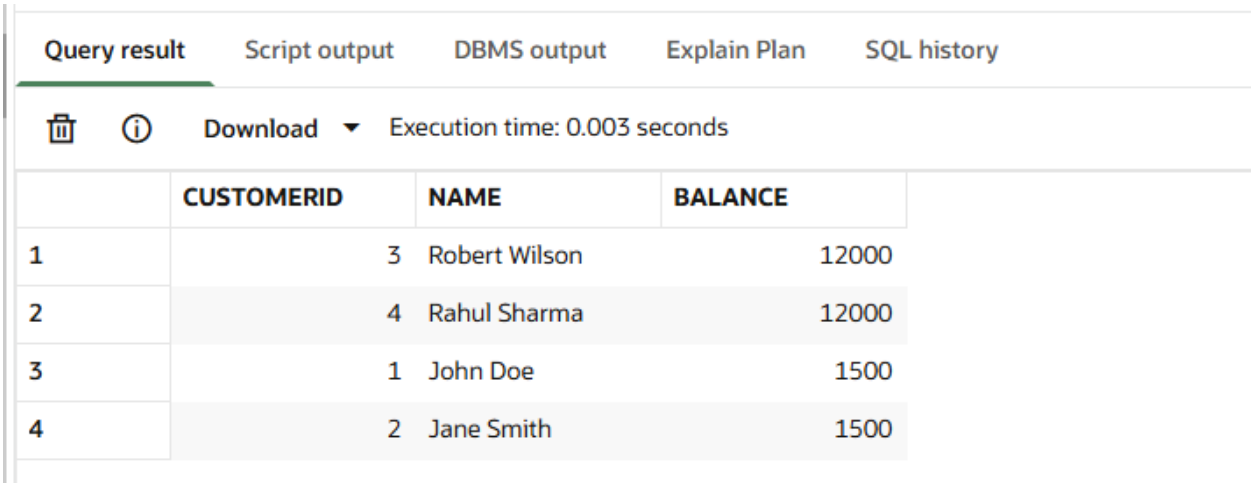
```
CREATE OR REPLACE PACKAGE CustomerManagement AS

  -- Procedure to add a new customer
  PROCEDURE AddCustomer (
    p_customer_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
  );
```

```
  -- Procedure to update customer details
  PROCEDURE UpdateCustomerDetails (
    p_customer_id NUMBER,
    p_name VARCHAR2,
    p_balance NUMBER
  );

  -- Function to get customer balance
  FUNCTION GetCustomerBalance (
    p_customer_id NUMBER
  ) RETURN NUMBER;

END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

  PROCEDURE AddCustomer (
    p_customer_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
  ) AS

  BEGIN

    -- Insert new customer details
    INSERT INTO Customers (
      CustomerID,
      Name,
      DOB,
      Balance,
      LastModified
    )
    VALUES (
      p_customer_id,
      p_name,
      p_dob,
      p_balance,
      SYSDATE
    );

    -- Save changes
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Added Successfully.');

  EXCEPTION

    -- Handle duplicate customer ID error
    WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Error: Customer already exists');
```

```
  END AddCustomer;


  PROCEDURE UpdateCustomerDetails (
    p_customer_id NUMBER,
    p_name VARCHAR2,
    p_balance NUMBER
  ) AS

  BEGIN

    -- Update customer details
    UPDATE Customers
    SET Name = p_name,
        Balance = p_balance,
        LastModified = SYSDATE
    WHERE CustomerID = p_customer_id;

    -- Save changes
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Details Updated Successfully.');

  END UpdateCustomerDetails;


  FUNCTION GetCustomerBalance (
    p_customer_id NUMBER
  ) RETURN NUMBER AS

    v_balance NUMBER;

  BEGIN

    -- Fetch customer balance
    SELECT Balance
    INTO v_balance
    FROM Customers
    WHERE CustomerID = p_customer_id;

    -- Return balance
    RETURN v_balance;

  END GetCustomerBalance;


END CustomerManagement;
/

BEGIN
  -- Call procedure to add a new customer
  CustomerManagement.AddCustomer(
    3,
    'Robert Wilson',
```

```
    TO_DATE('1995-06-10','YYYY-MM-DD'),
    5000
  );
END;
/

-- Display customer details
SELECT CustomerID,
       Name,
       Balance
FROM Customers;
```

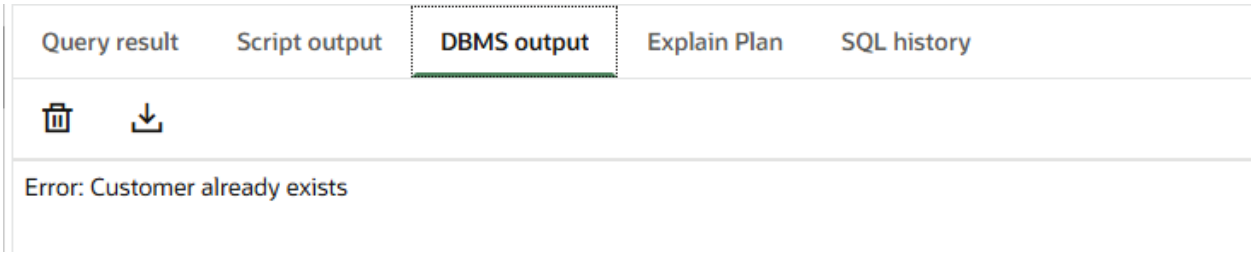
**OUTPUT:**



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

| | CUSTOMERID | NAME | BALANCE |
|---|---|---|---|
| 1 | 3 | Robert Wilson | 12000 |
| 2 | 4 | Rahul Sharma | 12000 |
| 3 | 1 | John Doe | 1500 |
| 4 | 2 | Jane Smith | 1500 |



Query result | Script output | DBMS output | Explain Plan | SQL history

Error: Customer already exists

## Scenario 2: EmployeeManagement

Package with procedures and a function for employee records.

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS

  -- Procedure to hire a new employee
  PROCEDURE HireEmployee (
    p_employee_id NUMBER,
    p_name VARCHAR2,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2
  );

  -- Procedure to update employee details
```

```
  PROCEDURE UpdateEmployeeDetails (
    p_employee_id NUMBER,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2
  );

  -- Function to calculate annual salary
  FUNCTION CalculateAnnualSalary (
    p_employee_id NUMBER
  ) RETURN NUMBER;

END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS


  PROCEDURE HireEmployee (
    p_employee_id NUMBER,
    p_name VARCHAR2,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2
  ) AS

  BEGIN

    -- Insert new employee details
    INSERT INTO Employees (
      EmployeeID,
      Name,
      Position,
      Salary,
      Department,
      HireDate
    )
    VALUES (
      p_employee_id,
      p_name,
      p_position,
      p_salary,
      p_department,
      SYSDATE
    );

    -- Save changes
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee Added Successfully.');

  EXCEPTION

    -- Handle duplicate employee ID
```

```
    WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Error: Employee already exists');

  END HireEmployee;



  PROCEDURE UpdateEmployeeDetails (
    p_employee_id NUMBER,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2
  ) AS

  BEGIN

    -- Update employee details
    UPDATE Employees
    SET Position = p_position,
        Salary = p_salary,
        Department = p_department
    WHERE EmployeeID = p_employee_id;

    -- Save changes
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee Details Updated Successfully.');

  END UpdateEmployeeDetails;



  FUNCTION CalculateAnnualSalary (
    p_employee_id NUMBER
  ) RETURN NUMBER AS

    v_monthly_salary NUMBER;

  BEGIN

    -- Fetch employee salary
    SELECT Salary
    INTO v_monthly_salary
    FROM Employees
    WHERE EmployeeID = p_employee_id;

    -- Return annual salary
    RETURN v_monthly_salary * 12;

  END CalculateAnnualSalary;


END EmployeeManagement;
/
```

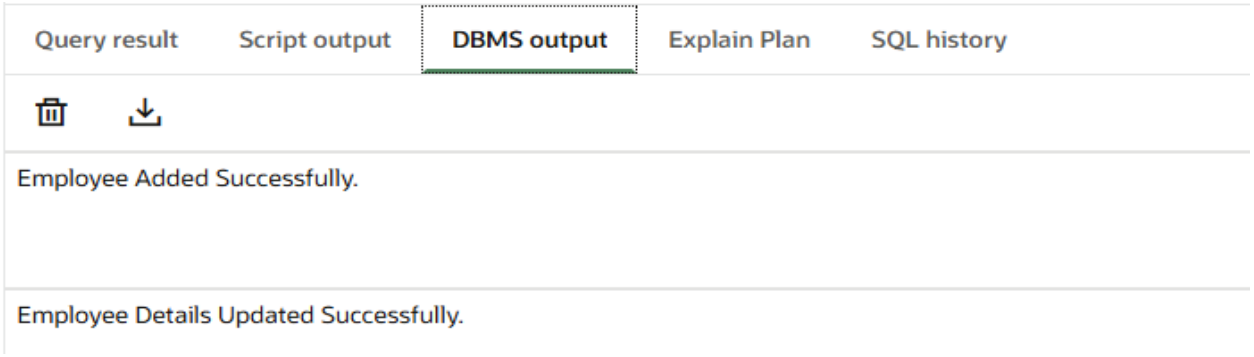
```
-- Test HireEmployee procedure
BEGIN
  EmployeeManagement.HireEmployee(
    3,
    'David Smith',
    'Analyst',
    50000,
    'Finance'
  );
END;
/

-- Test UpdateEmployeeDetails procedure
BEGIN
  EmployeeManagement.UpdateEmployeeDetails(
    3,
    'Senior Analyst',
    60000,
    'Finance'
  );
END;
/

-- Test CalculateAnnualSalary function
SELECT EmployeeID,
       Name,
       Salary,
       EmployeeManagement.CalculateAnnualSalary(EmployeeID) AS Annual_Salary
FROM Employees;
```

**OUTPUT:**

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.009 seconds

| | EMPLOYEEID | NAME | SALARY | ANNUAL_SALARY |
|---|---|---|---|---|
| 1 | 1 | Alice Johnson | 84700 | 1016400 |
| 2 | 2 | Bob Brown | 60000 | 720000 |
| 3 | 3 | David Smith | 60000 | 720000 |




## Scenario 3: AccountOperations

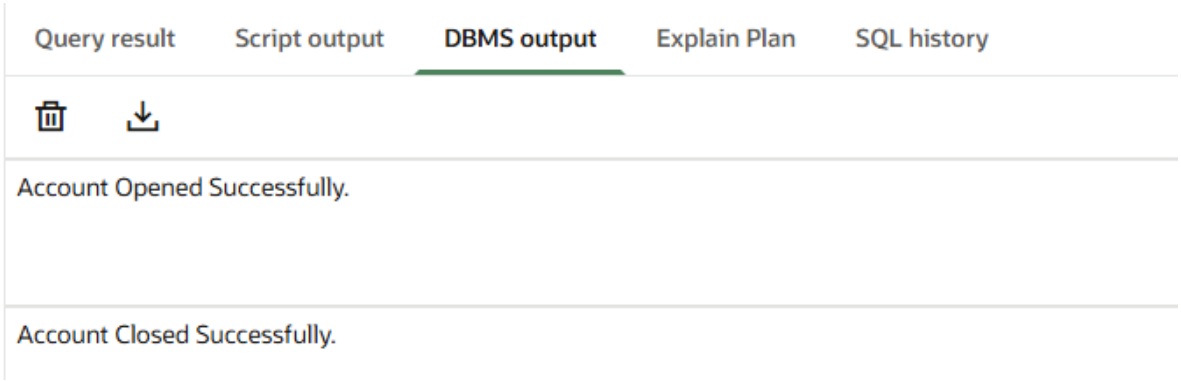
Package with procedures and a function for account operations.

```
CREATE OR REPLACE PACKAGE AccountOperations AS

  -- Declare procedures and function for account operations
  PROCEDURE OpenAccount (
    p_account_id NUMBER,
    p_customer_id NUMBER,
    p_account_type VARCHAR2,
    p_balance NUMBER
  );

  PROCEDURE CloseAccount (
    p_account_id NUMBER
  );

  FUNCTION GetTotalBalance (
    p_customer_id NUMBER
  ) RETURN NUMBER;

END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

  PROCEDURE OpenAccount (
    p_account_id NUMBER,
    p_customer_id NUMBER,
    p_account_type VARCHAR2,
    p_balance NUMBER
  ) AS

  BEGIN

    -- Insert a new account record
    INSERT INTO Accounts (
      AccountID,
      CustomerID,
      AccountType,
      Balance,
```

```
    LastModified
  )
  VALUES (
    p_account_id,
    p_customer_id,
    p_account_type,
    p_balance,
    SYSDATE
  );

  -- Save changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Account Opened Successfully.');

EXCEPTION

  -- Handle duplicate account ID error
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Error: Account already exists');

END OpenAccount;


PROCEDURE CloseAccount (
  p_account_id NUMBER
) AS

BEGIN

  -- Delete the account record
  DELETE FROM Accounts
  WHERE AccountID = p_account_id;

  -- Save changes
  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Account Closed Successfully.');

EXCEPTION

  -- Handle errors and rollback transaction
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error closing account: ' || SQLERRM);

END CloseAccount;


FUNCTION GetTotalBalance (
  p_customer_id NUMBER
) RETURN NUMBER AS

  v_total NUMBER;
```

```
  BEGIN

    -- Calculate total balance of customer accounts
    SELECT SUM(Balance)
    INTO v_total
    FROM Accounts
    WHERE CustomerID = p_customer_id;

    -- Return total balance
    RETURN NVL(v_total, 0);

  END GetTotalBalance;


END AccountOperations;
/

--Test OpenAccount
BEGIN
  -- Open a new account for customer ID 1
  AccountOperations.OpenAccount(7, 1, 'Savings', 5000);
END;
/

--Test CloseAccount
BEGIN
  -- Close account with Account ID 3
  AccountOperations.CloseAccount(3);
END;
/

--Test GetTotalBalance
SELECT AccountOperations.GetTotalBalance(1) AS Total_Balance
FROM DUAL;
```

**OUTPUT:**



Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

| | TOTAL_BALANCE |
|---|---|
| 1 | 10616.08 |